PES UNIVERSITY, Bangalore

Established under Karnataka Act No. 16 of 2013)

Department of Computer Science & Engineering

UE24CS251A: DIGITAL DESIGN AND COMPUTER ORGANIZATION

Unit 4

1. Derive the expression for the total number of AND gates and Full Adders in an n × n array multiplier.
   Answer:
   Each of the n bits of the multiplicand is multiplied with n bits of the multiplier.
   Number of AND gates=$n \times n = n^2$
   Each additional partial-product row (except the first) requires a set of Full Adders to add to the accumulated sum.
   Number of Full Adders=$n \times (n-1)$
   Example for n = 4:
   AND gates=16, Full Adders=12
   Hence, **a 4×4 array multiplier uses 16 AND gates and 12 Full Adders.**

2. **Explain how the array multiplier implements binary multiplication using digital logic.**

Answer:

- Each cell contains an **AND gate** and a **Full Adder**.
- The AND gate performs the product mj·qim_j·q_imj·qi.
- The Full Adder adds this product bit with the previous **Partial Product (PP)** and the **Carry-in**.
- The sum output forms the new partial product bit, and carry propagates diagonally to the next cell.
- The multiplicand bits shift downward, and the multiplier bits feed horizontally across the array.
  This systematic flow enables **parallel generation and sequential accumulation** of partial products, achieving high-speed multiplication through a structured hardware array.

3.Perform binary multiplication using the Shift–Add algorithm for M = 1011 (11) and Q = 1101 (13).

Ans:

Cycle	q_0	Action	A (after add)	(C,A,Q) >> 1 → New A	New Q
1	1	A ← A+M	1011	0101	1110
2	0	No Add	—	0010	1111
3	1	A ← A+M	1101	0110	1111
4	1	A ← A+M	1101 (+carry)	1000	1111

4. What is Booth's Algorithm? Explain its operation with bit transitions (Q_0, Q_{-1}) and show how +1, –1, and 0 operations are determined

Ans:

Idea. Scan the multiplier right→left tracking the pair (Q_0, Q_{-1}).

- When we see a **0→1 transition**, select **−1 × shifted M** (i.e., subtract M).
- When we see a **1→0 transition**, select **+1 × shifted M** (i.e., add M).
- Runs of 1s collapse to at most two operations (entering and leaving the run), cutting additions.

Operational table (for accumulator A):

- (Q_0,Q_{-1}) = **00** → no add/sub; shift
- (Q_0,Q_{-1}) = **01** → **+1 × M** (add M)
- (Q_0,Q_{-1}) = **10** → **−1 × M** (subtract M)
- (Q_0,Q_{-1}) = **11** → no add/sub; shift

5. For a multiplier pattern 01111110, demonstrate why it represents the best case for Booth's algorithm. For 010101010, show why it is the worst case (maximum transitions).

Ans: **Best case: 01111110**

Interpret the bit transitions while scanning right→left:

- There's a **single 01** at the **right boundary of the 1-run** → one **+1 × M**.
- There's a **single 10** at the **left boundary of the 1-run** → one **−1 × M**.
- Inside the block of 1s you get **11** repeatedly → **no add/sub**.
  Net effect: the long run of 1s collapses to **just two operations** (+M at one edge and

−M at the other). That's minimal non-zero operations, hence "best case." The slides show this as the good (best) example.

Worst case: 010101010 (alternating 0s and 1s)

Every adjacent pair flips, so almost **every step is a 01 or 10**, meaning **you select a summand at each bit** (alternating +M and −M):
010101010 → +1, −1, +1, −1, …
This maximizes the number of adds/subtracts—the classic Booth worst case.

6. **Explain the Restoring Division Algorithm with steps and an example.**

Answer:

Concept:
Restoring Division is a hardware-based binary division method similar to longhand subtraction. It repeatedly subtracts the divisor from the partial remainder (A) and restores it if the subtraction goes negative.

Algorithm Steps:

1. Initialize registers
    - **M** = Divisor (n bits)
    - **Q** = Dividend (n bits)
    - **A** = 0 (n + 1 bits to hold remainder and sign)
2. **Repeat n times:**
   i. Left-shift the pair (A,Q) by 1 bit.
   ii. Perform subtraction: A = A − M.
   iii. If the sign bit of A is 1 (negative),
   → Set $q_0 = 0$, and restore: A = A + M.
   Else set $q_0 = 1$.
3. After n iterations:
    - **A** contains the *remainder*.
    - **Q** contains the *quotient*.

7. Explain the Non-Restoring Division Algorithm and how it improves efficiency.

Ans:

Concept:
The non-restoring algorithm eliminates the extra "restoration" addition step. Instead, it decides the next operation (add or subtract) based on the current sign of A.

Algorithm Steps:

1. Initialize
    - M = Divisor, Q = Dividend, A = 0 (n + 1 bits).
2. **Repeat n times:**
   i. Left-shift (A,Q) by 1.
   ii. If $A \geq 0 \rightarrow A = A - M$.
   If $A < 0 \rightarrow A = A + M$.
   iii. If $A \geq 0 \rightarrow q_0 = 1$; Else $q_0 = 0$.
3. After n iterations:
   If $A < 0 \rightarrow A = A + M$ (to restore final remainder)

Advantages:

- Saves one addition per unsuccessful subtraction cycle.
- Faster since "restoration" is replaced by conditional add/sub logic.

8. **Differentiate between Restoring and Non-Restoring Division Algorithms.**

Ans:

Feature	**Restoring Division**	**Non-Restoring Division**
Restoration Step	Required after every negative subtraction	Not required; next operation compensates
Control Logic	Slightly simpler	Slightly more complex (conditional add/sub)
Speed	Slower (extra add)	Faster (saves one add per iteration)
No. of Operations	2 × n (add/sub) ≈ higher	≈ n (add/sub) ≈ lower
Remainder Adjustment	Automatically handled	Final restoration if negative
Example Result (6 ÷ 3)	010 remainder 000	Same result but fewer cycles

9. **For n-bit operands, analyze the time complexity of restoring vs. non-restoring division.**

Answer:

Each bit requires a full iteration involving:

- **Restoring:** 1 shift + 1 subtraction + possible restoration (add).
- **Non-Restoring:** 1 shift + 1 conditional add/sub.

Hence,

- **Restoring → ≈ 2n operations.**
- **Non-Restoring → ≈ n operations.**

Conclusion: Non-Restoring Division reduces ALU operations by ≈ 50% and is preferred for high-speed digital processors.

10. **Explain how sign handling is performed in binary division of signed numbers.**

Ans:

Both restoring and non-restoring methods work on **unsigned or positive signed** magnitudes. For **signed operands**, perform these steps:

1. Determine sign of result: **Sign(Quotient) = Sign(Dividend) ⊕ Sign(Divisor)**.
2. Take magnitudes (absolute values) of both numbers.
3. Apply restoring/non-restoring division.
4. Assign sign obtained in step 1 to the quotient.
5. The remainder has the same sign as the dividend.

11. **Explain the concept of Floating Point Representation.**

Answer:
Floating point representation is used to express real numbers that are too large or too small to be represented in fixed-point format. It follows the scientific notation:

$\pm M \times B^E$

where **M** is the mantissa (or significand), **B** is the base (usually 2 for binary systems), and **E** is the exponent.
The IEEE-754 standard defines three parts:

- **Sign bit (S):** 0 for positive, 1 for negative
- **Exponent bits (E):** Represent the range of values
- **Mantissa bits (M):** Represent precision (fractional accuracy)

For example, in 32-bit single precision:

- 1 bit → sign
- 8 bits → exponent (with bias 127)
- 23 bits → mantissa

12. What are the differences between Fixed Point and Floating Point representation?

Answer:

Feature	Fixed Point	Floating Point
Decimal Point	Fixed location	Varying (floating)
Range	Limited	Very wide
Accuracy	Low	High
Representation	Integer + fractional bits predefined	Mantissa + exponent
Example	0110.1100 = 6.75	$1.101 \times 2^{-3} = 0.231$

Fixed point is efficient for hardware arithmetic, but floating point is preferred in scientific and engineering applications where precision and dynamic range are essential

13. **What are Floating Point Special Cases?**

Answer:

- **Zero:** Exponent = 0, Mantissa = 0
- **Infinity:** Exponent = all 1s, Mantissa = 0
- **NaN (Not a Number):** Exponent = all 1s, Mantissa ≠ 0 (e.g., $\sqrt{-1}$, divide by zero)
- **Denormalized Numbers:** Exponent = 0, Mantissa ≠ 0 (used for representing very small numbers close to zero)

14. **What are Floating Point Rounding Modes?**

Answer:
IEEE-754 defines four rounding modes:

1. **Round down (toward $-\infty$)**
2. **Round up (toward $+\infty$)**
3. **Round toward zero**
4. **Round to nearest (default)**

Example: Round 1.100101 (1.578125) to 3 fraction bits:

- Down → 1.100
- Up → 1.101
- Toward zero → 1.100
- To nearest → 1.101 (since 1.625 is closer than 1.5)

15. **Explain the working of a register bit implemented using a D flip-flop, multiplexer, and tri-state gate.**

Answer:
A single bit of a processor register (say, Ri) can be implemented using:

- **Edge-triggered D flip-flop** for data storage
- **Multiplexer (MUX)** for data input selection
- **Tri-state gate** for bus output control

The **multiplexer** selects between two inputs:

1. **Bus data** (when Riin=1) — loads new data from the bus into the flip-flop.
2. **Feedback path** (when Riin=0) — retains the existing stored value.

At the **rising edge of the clock**, the D flip-flop stores the selected value.
The **tri-state gate** connects the output Q to the common bus only when Riout=1; otherwise, the output is in the **high-impedance (open-circuit)** state, preventing interference on the bus.
This mechanism allows controlled data transfer between registers in a synchronous processor.

16. **What are the control signals used for register transfer operations? Explain their purpose with an example.**

Answer:
Each register in a processor has two control signals:

- **Rout (Register Output Enable)** — places the register's contents onto the internal bus.
- **Rin (Register Input Enable)** — loads data from the bus into the register.

Example:
To transfer data from R1to R2

1. Set R1outin=1contents of R1 appear on the bus.
2. Set R2in=1 data on the bus is loaded into R2.

After the next clock edge, the data transfer is complete, and both signals return to 0.
This mechanism forms the foundation of **register-to-register** and **register-to-memory** data flow in digital

17. **explain the functional roles of MAR, MDR, and internal processor bus during instruction execution.**

Answer:

- **Memory Address Register (MAR):**
  Holds the address of the memory location involved in a read or write operation. It interfaces between the internal processor bus and the external memory bus.
- **Memory Data Register (MDR):**
  Temporarily holds the data being transferred to or from memory.

- During a **read**, MDR receives data from the memory bus.
- During a **write**, MDR provides data from the processor to memory.

- **Internal Processor Bus:**
  A shared data pathway connecting the ALU, registers, MAR, MDR, and control unit. It enables efficient data transfer between internal processor components without requiring multiple dedicated connections.

Together, these elements support **memory fetch and store cycles** in the instruction execution process.

18. **explain the control sequence for executing a complete instruction in a single-cycle datapath.**

Answer:
Instruction execution proceeds in several control steps, each corresponding to a clock cycle:

1. **Step 1 – Instruction Fetch:**
   The contents of the Program Counter (PC) are sent through the ALU (R=B), loaded into the Memory Address Register (MAR), and a **Read** command is issued. Simultaneously, the Incrementer unit adds 4 to the PC.
2. **Step 2 – Wait for MFC:**
   The processor waits for the **Memory Function Complete (MFC)** signal, indicating data is available from memory.
3. **Step 3 – Load IR:**
   The instruction received from memory (stored in MDR) is transferred to the **Instruction Register (IR)**.
4. **Step 4 – Execute Phase:**
   The operation specified by the IR is performed (e.g., Add, Branch, or Load/Store).

Thus, in a single-cycle datapath, fetch, decode, and execute phases are closely synchronized, improving pipeline efficiency.

19. **explain how a branch instruction is executed, and differentiate between unconditional and conditional branches.**

Answer:
A **branch instruction** changes the normal sequential flow of execution by loading a new address into the PC.

- The **branch target address** is obtained by adding an **offset (X)** to the address of the instruction following the branch.
- For example, if the branch instruction is at 2000 and the target is 2050, then **X = 46** (since next address = 2004, 2050 − 2004 = 46).

Unconditional Branch:
Always replaces the PC with the target address — no condition testing.

Conditional Branch:
Before updating the PC, the **status flags** (e.g., N, Z, C) are checked.

20. **What is Multiple Bus Organization? How does it improve data transfer speed within the processor?**

Answer:
The **Multiple Bus Organization** enhances data transfer efficiency by using multiple internal buses:

- Two **output buses (A, B)** carry data simultaneously from two different registers.
- One **input bus (C)** allows writing data back to a third register in the same clock cycle.
- Together, these form a **three-port register file**.

Advantages:

- Enables parallel operand fetching (R1, R2 → ALU).
- Reduces number of control cycles.
- Facilitates high-throughput instruction execution (e.g., Add R4, R5, R6 executed in one cycle).

Example:
Bus A → R5, Bus B → R6, Bus C → R4 (destination).

21. Explain the working of a Hardwired Control Unit. What are its key components and control signals?

Ans:

Answer:
A **hardwired control unit** uses combinational logic to generate control signals directly from hardware circuits rather than from a program.
It operates based on three main inputs:

- **Control step counter:** Tracks the current micro-operation step.
- **Instruction register (IR):** Determines which instruction is being executed.
- **Condition codes and external inputs:** Include flags such as Zero, Carry, Negative, and external signals like **MFC (Memory Function Completed)** or interrupts.

Each step in instruction execution corresponds to one clock cycle. The **RUN** signal controls progression through the steps — when RUN = 1, the counter increments; when RUN = 0, it halts (for example, during memory wait). The **END** signal resets the step counter at the end of each instruction cycle to start fetching the next instruction.

Internally, the control unit consists of a **decoder/encoder structure** that produces signals such as Yin, PCout, Add, and Zin. It behaves as a **finite state machine (FSM)**, where each state corresponds to one control step. Hardwired control is **fast and efficient** but **difficult to modify**, making it suitable for fixed-instruction processors.

22. Compare and contrast Hardwired Control and Microprogrammed Control.

Ans:

Feature	Hardwired Control	Microprogrammed Control
Implementation	Uses combinational logic and flip-flops	Uses microinstructions stored in control memory
Speed	Very fast	Slower due to memory fetch overhead
Flexibility	Difficult to modify; changes require rewiring	Easy to modify by altering control memory contents
Complexity	High for complex instruction sets	Easier to implement complex instructions
Control Signal Generation	Direct hardware decoding	Generated from control words (CWs)
Usage	RISC-type processors	CISC and complex instruction sets

23. Explain the concept of Microprogrammed Control. What is a Control Word (CW)?

Ans:

In a **microprogrammed control unit**, control signals are generated using a **microprogram** — a small program stored in the control memory. Each instruction's execution is defined as a sequence of **microinstructions**, and each microinstruction corresponds to one **control word (CW)**.

A **Control Word** is a binary word in which each bit represents a specific control signal (e.g., PCout, Yin, Add, Zin, etc.). During execution, each CW activates the necessary signals in parallel to perform one micro-operation.

Microprogrammed control supports **conditional branch microinstructions**, allowing decision-making based on condition flags or instruction bits. The **microprogram counter (uPC)** keeps track of the address of the next microinstruction. It increments sequentially unless a branch or end condition is encountered.

Advantages:

- Easier modification (just change microcode).
- Simplifies design for complex instruction sets.

Disadvantage:

- Slightly slower due to microinstruction memory fetch latency.

24. Distinguish between Horizontal and Vertical Microprogramming.

Answer:
Horizontal Microprogramming:

- Each microinstruction directly specifies all control signals with minimal or no encoding.
- Multiple control signals can be activated simultaneously.
- Offers **high speed** and **parallelism** but uses **long microinstructions** (wide control words).
- Example: 20-bit control word for 42 signals.

Vertical Microprogramming:

- Uses **high encoding** of control signals — groups mutually exclusive signals and represents them with binary codes.
- Produces **shorter microinstructions**, saving memory space but reducing parallelism.
- Execution is **slower** due to additional decoding logic.

Trade-off:
Horizontal organization favors performance (used in high-speed processors), whereas vertical organization favors compactness (used where memory space is limited).